

KVMillésime - La part des Anges

Gist

Comme les deux autres challenges KVMillésime, c'est un challenge d'ingénierie de la crypto. Sur ce système, une mauvaise gestion de la mémoire – du cache d'accélération d'accès aux zones physiques de la mémoire – permet la corruption d'une signature, et l'extraction de la clef de signature. La crypto mathématique tourne sur des machines réelles :).

Préambule technique : Mécanisme de virtualisation de la mémoire

Promis, c'est utile. Ça rendra la lecture suivante plus facile. Au pire, vous aurez appris des choses.

Ici et dans la suite, j'appelle *hyperviseur* le code du dossier hyperviseur. C'est un manager de VM. Pensez à VMware Workstation ou VirtualBox. Ou "hyperviseur-du-pauvre" dans ce challenge. C'est l'autorité absolue pour toutes les VMs qu'il fait tourner. Il est responsable de gérer les erreurs (processeur, mémoire, etc) que la VM pourrait lever pendant son exécution. Le *guest* est cette VM. Son code se trouve dans le dossier `guest/`.

En virtualisation, l'hyperviseur cherche à être le plus invisible possible aux yeux du guest. Le guest s'attend à avoir de la mémoire virtuelle et à ne pas avoir à traiter avec des adresses physiques de mémoire ? L'hyperviseur prend le relais et lui donne les mappings. Le schéma ci-dessous montre ce que voit le guest, et les conversions réalisées en sous-main soit par l'hyperviseur, soit par le processeur.

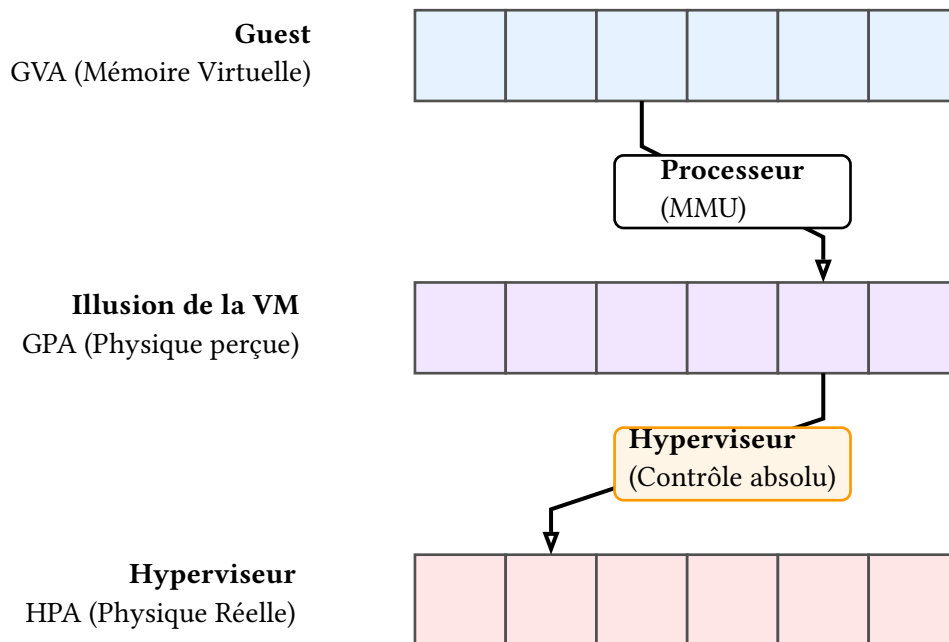


Figure 1: Résolution des adresses mémoire : de la VM au matériel physique

En pratique, résoudre une adresse virtuelle, *ie.* remonter la chaîne GVA (Guest Virtual Address) - GPA (Guest Physical Address) - HVA (Hypervisor Virtual Address), est lent. Le CPU introduit le *Translation Lookaside Buffer* (TLB) pour enregistrer le mapping résultant des quelques dernières résolutions GVA - HPA. Si l'adresse à résoudre est dedans, le CPU ne fait pas la résolution et prend directement l'adresse physique cible.

Problème : si l'hyperviseur change le mapping mémoire GPA - HPA¹, la TLB est obsolète. Si le guest la garde telle quelle, il se rendra sur des pages (physiques) qui ne lui appartiennent plus, et ne lira pas les données les plus à jour. Il existe donc plusieurs méthodes pour invalider le cache TLB. Soit des instructions processeur (changer le registre cr3, invlpg), ou comme ici, une instruction KVM côté hyperviseur qui gère l'administratif du mapping mémoire `ioctl(., KVM_SET_USER_MEMORY_REGION, .)`.

Donc en résumé, si l'hyperviseur change les mappings mémoire du guest, soit il le fait proprement avec `KVM_SET_USER_MEMORY_REGION`, soit il force l'invalidation du cache TLB du guest. Sinon, le guest aura un franken-TLB, où les caches anciens mènent vers de la mémoire obsolète, et les caches nouvellement résolus sont bons.

Solution

Exploration du code

Vision générale

Une lecture de haut niveau donne les idées générales du code : c'est (encore) un système virtualisé, avec du KVM. On a un *client* (dossier `client/`), un hyperviseur et un guest.

Concernant l'hyperviseur :

- son code est localisé dans le dossier aptement nommé `hypervisor/` ;
- il attend des connexions réseau, `websocket` d'après le code de `client/` ;
- il accepte plusieurs connexions, et des commandes venant du client. Ces commandes sont gérées par la fonction `_execute_client_command` dans `network.c`.

Concernant le guest :

- son code est dans `guest/`, et c'est du code bare-metal ;
- il fait principalement des "trucs crypto", probablement du RSA si le nom des fichiers est correct ;
- comme c'est du bare-metal, on a le droit à une réimplémentation d'une bibliothèque `big-num`.

`client/` n'est qu'un front-end web, on n'apprendra rien qu'on n'ait pas déjà dans le code C.

Localiser le flag

La lecture de `vm.c` (ou un `grep`) donne le chemin critique à atteindre : `_emit_flag_to_client`, `_dispatch_guest_response`, `_handle_port_output`, `_dispatch_io_event`, `_vcpu_worker_thread`.

```
static void _dispatch_guest_response(void) {
    // ...
    if (state.output_ptr == 1 && state.output_buf[0] == STATUS_ACCESS_GRANTED) {
        _emit_flag_to_client(client_fd);
    }
    // ...
    state.output_ptr = 0;
}

static void * _vcpu_worker_thread(void *arg) {
    while (1) {
        if (ioctl(state.vcpu_fd, KVM_RUN, 0) < 0) {
            if (errno == EINTR) continue;
            break;
        }
        // Arriver ici signifie que l'exécution du guest s'est interrompue.
    }
}
```

¹En vrai, ce serait HVA, et on aurait un nouvel étage HVA-HPA. Mais on ne va pas complexifier avec une nouvelle couche d'indirection

```

        if (state.run->exit_reason == KVM_EXIT_IO) {
            // Donc, si on est ici, le guest a écrit (avec l'instruction assembleur
            `out`) sur un port quelconque.
            _dispatch_io_event();
        } else if (state.run->exit_reason == KVM_EXIT_SHUTDOWN) {
            exit(0);
        }
    }
    return NULL;
}

```

La validation pour obtenir le flag doit venir du guest, et il doit renvoyer STATUS_ACCESS_GRANTED. Côté guest, on trouve :

```

static void _handle_withdraw_funds(void) {
    // ...

    recv_bytes(msg, 32); // Recv message and signature
    recv_bytes(sig, 128);

    // validate messages
    int msg_match = 1;
    for (int i = 0; i < 32; i++) {
        if (msg[i] != target_msg[i]) msg_match = 0;
    }

    if (!msg_match) {
        outb(PORT_DATA, STATUS_ACCESS_DENIED);
    } else {
        if (rsa_verify(msg, sig, g_params)) {
            outb(PORT_DATA, STATUS_ACCESS_GRANTED);
        } else {
            outb(PORT_DATA, STATUS_BAD_SIGNATURE);
        }
    }
    outb(PORT_CMD, SIGNAL_DONE);
}

```

A priori, si on peut croire le code, il faut envoyer au guest une signature pour la chaîne "ADMIN_RUGPULL_ROI_1000X". Enfin un peu de crypto.

Le reste du code indique les opérations naïvement à notre disposition :

- obtenir la clef publique ;
- obtenir la signature d'un message choisi, mais différent du token à trouver.

Chemin d'attaque de la signature

En parcourant le code du guest, on arrive à

```

void rsa_crt_sign(const uint8_t *msg, uint8_t *sig, const struct rsa_params *params,
const struct q_params *q_p) {
    _isolate_prime_constants(params);

    // padding, pour bloquer les attaques malines sur l'homomorphisme du RSA ;)
    // Calcul de S_p = m^(d_p) [p]
    // ...

    _wait_for_secure_element_sync();
}

```

```

// Calcul de  $S_q = m^{(d_q)} [q]$ 
// ...
// Recombinaison avec la formule de Gartner

for (int i = 0; i < 128; i++) {
    sig[i] = (uint8_t)(_s_full.limbs[i / 4] >> ((i % 4) * 8));
}

static void _wait_for_secure_element_sync(void) {
    outb(PORT_SECURE_ELEMENT, 1);
    while (inb(PORT_SECURE_ELEMENT) == 0) {
        if (inb(PORT_STATUS) == 1) {
            handle_maintenance(inb(PORT_DATA));
        }
    }
}

```

La signature est interrompue entre le calcul de $S_p \bmod p$ et $S_q \bmod q$ et donne la main à l'hyperviseur pour une attente active de 1 sec. Si on sait modifier les paramètres de calcul de S_q pendant l'attente active, on pourra utiliser l'attaque de Bellcore (cf. plus bas pour la description).

Vulnérabilité : injection de faute

C'est dans le nom du challenge : "KVM". Et développer un code bare-metal ou un hyperviseur custom, c'est lourd. Il est probablement la source du problème. Deux points sont critiques pour un hyperviseur, en particulier custom : la phase d'initialisation du CPU d'une part, la gestion de la mémoire de l'autre. Ici, on fait des choses non-standard. `protocol.h` met l'effort de séparer les variables nécessaires à RSA en deux morceaux distincts.

```

#define CONFIG_REGION_ADDR    0x7000    // Static RSA params (p, dp, qInv, n, e)
#define SCRATCHPAD_VIRT_ADDR  0x8000    // Volatile RSA params (q, dq)

```

Donc, possiblement, en fonction de comment le code gère ces morceaux, on pourrait avoir une désynchronisation entre ces deux éléments de clef – soit par un TOCTOU, soit par une mauvaise gestion mémoire.

Et en effet, on trouve un problème dans la mal-nommée `_shift_reality_plane`.

```

static void _shift_reality_plane(uint32_t virtual_address, uint32_t physical_frame) {
    uint32_t *dma_ring = (uint32_t *) (state.mem_low + PAGE_TABLE_ADDR);
    uint32_t ring_index = virtual_address / SCRATCHPAD_SIZE;

    dma_ring[ring_index] = physical_frame | PTE_PRESENT_WRITABLE_USER;
}

```

Elle est appelée dans

```

void update_session_context_mapping(struct rsa_params *new_params) {
    uint8_t *next_c_ptr = // Un pointeur dans la mémoire du host
    uint32_t next_c_gpa = // Une adresse physique du point de vue du guest

    uint8_t *next_q_ptr = // Un pointeur dans la mémoire du host
    uint32_t next_q_gpa = // Une adresse physique du point de vue du guest

    memcpy(next_c_ptr, new_params, sizeof(struct rsa_params));
    memcpy(next_q_ptr, &state.real_q_params, sizeof(struct q_params));

    _shift_reality_plane(CONFIG_REGION_ADDR, next_c_gpa);
}

```

```

    _shift_reality_plane(SCRATCHPAD_VIRT_ADDR, next_q_gpa);

    state.active_page_idx = 1 - state.active_page_idx;
}

```

update_session_context_mapping copie du contenu dans next_{c,q}_ptr, puis change le mapping du guest. C'est exactement ce qu'on mentionnait dans la première section : on fait des modifications de mapping sans passer par le ioctl KVM adéquat, et sans invalider le cache.

Le guest va se trouver dans un état instable où, tant qu'il n'aura pas fait trop de résolutions mémoire (anciennes valeurs du cache TLB), ses accès mémoires se feront sur l'ancien mapping. Au moment où le cache de TLB s'est correctement réajusté, il verra les nouvelles valeurs.

Et justement, la commande CMD_RESEED_PRNG, accessible tout le temps, fais suffisamment de lectures, suffisamment éloignées pour forcer un flush complet (*ie.* remplir toutes les cases par de nouvelles entrées) du cache TLB :

```

static uint32_t _mix_ambient_ram_noise(void) {
    // ...

    for (int i = 0; i < ENTROPY_WIDTH; i++) {
        // Lecture page par pages, suffisamment de fois pour remplir le cache TLB
        uint32_t *noise_ptr = (uint32_t *) (HEAP_BASE + (i * 4096));
        // ...
        // PRNG bullshit jamais utilise
        ///
    }
    // ...
}

```

Ainsi on a une macro pour forcer une erreur pendant le calcul de signature RSA-CRT :

- demander une re-génération de clef, mais qui n'est pas effective dans l'immédiat ;
- faire signer un message avec une moitié de clef RSA-CRT ;
- forcer une mise à jour du cache TLB pendant la pause, changeant le mapping mémoire du guest, et donc la valeur de ses constantes de calcul ;
- et faire signer une deuxième avec une deuxième moitié de clef, mais différente.

C'est un cas d'école de l'*attaque de Bellcore*, l'objet de la section suivante.

Attaque de Bellcore

La signature utilise la formule de Garner pour optimiser le calcul *via* les Théorème des Restes Chinois (d'où le nom, CRT). Une clef privée est composée de $(p, q, q^{\{Inv\}}q^{\{-1\}} \bmod p, d_p = d \bmod (p-1), d_q = d \bmod (q-1))$. Une clef publique de (N, d) . Il y a deux étapes pour le calcul. Si une erreur matérielle vient changer la clef ou casser les valeurs entre les deux étapes, on peut en déduire un facteur de la clef publique, et donc la factoriser, et donc casser le système cryptographique.

Dans une situation normale, pour chiffrer m , on calcule

- $s_p = m^{\{d_p\}} \bmod p, s_q = m^{\{d_q\}} \bmod q$;
- $h = (s_p - s_q) \times q^{\{-1\}} \bmod p$ le coefficient de Garner (le coeff des CRT);

Et on renvoie $S = s_q + h \times q$ la signature.

Ici, on sait produire une injection de faute pendant le calcul de S_q .

On a donc $S_p = m^{\{d_{p_1}\}} \bmod p_1$ calculé avec les éléments de clefs $(p_1, d_{p_1}, q_1, d_{q_1}, q\text{Inv}_1)$. Injection de faute, $S_q = m^{\{d_{q_2}\}} \bmod q_2$, pour la clef $(p_2, d_{p_2}, q_2, d_{q_2}, q\text{Inv}_2)$. Le coefficient de Garner est calculé avec les constantes de la clef 1 : $h = (s_p - s_q) \cdot q\text{Inv}_1 \bmod p_1$. Et la signature finale $S = s_q + h \times q_2$.

Or, on remarque

$$S \bmod q_2 = s_q + h \times q_2 \bmod q_2 = s_q \bmod q_2$$

Ainsi

$$\begin{cases} S^e = m \bmod q_2 \\ S^e \neq m \bmod p_2 \end{cases}$$

Donc $\gcd(S^e - m, N_2) = q_2$: on peut factoriser N_2 et forger des signatures valides.

En bref

Le guest s'initialise. Il se génère un jeu de clefs.

- On demande la signature d'un élément, pour s'assurer que le guest a lu ses clefs et que le cache TLB contient ces résolutions mémoires ;
- On envoie une commande de re-gen de clef. Parce que le code de l'hyperviseur ne mets pas à jour le cache TLB, le guest continue à utiliser l'ancienne clef ;
- On demande la signature d'un nouveau message ;
- Pendant la pause de synchronisation, on envoie la commande de lecture d'entropie, qui nettoie complètement le cache TLB ;
- Le guest a ainsi signé la deuxième moitié du message avec la nouvelle clef. La signature est incohérente ;
- On extrait la clef privée par la signature avec l'attaque de Bellcore ;
- On signe le message de retrait du flag et des fonds.

Note : C'est une race, et ça joue avec une feature hardware. Le script de solution ne marchera pas à tous les coups. Il faudra probablement le relancer deux ou trois fois.